

GraphOne / FrontierAtlas — Architecture Document

Scope: 3-day take-home trial. This run collected real records from 5 live, first-party APIs/feeds (YC/Algolia, arXiv, GitHub, 5 RSS news feeds, RemoteOK + 4 job-board APIs/feeds), at the brief's full stated targets for Startups/Products/Research Papers (1,000 each). This document argues how the same architecture scales further to the brief's full 500k+ target, and documents the real numbers this run actually produced.

Entity	Brief's target	This run
Startups	1,000	1,000
Products	1,000	1,000
Research Papers	1,000	1,000 (321 GitHub-matched, 679 honest null)
Jobs (24h fresh)	uncapped	223 (out of ~413 raw across RemoteOK + 4 boards)
News (24h fresh)	uncapped	3 (out of 66 raw across all 5 feeds)

The News count is genuinely small, not under-filtering: checked at write time, only TechCrunch had anything under 24h old (newest 15.3h); The Verge's newest was 26.9h (just outside the window), Ars Technica and MIT Tech Review were 40+h stale, and VentureBeat intermittently returns zero entries under its WAF. A quiet 24h window across 5 outlets is a real characteristic of this run, not a bug — see README.md. Jobs/News are uncapped by the brief ("all found" within 24h), so neither was re-targeted when Startups/Products/Papers were scaled up to 1,000.

Per the brief's own instruction, this trial deliberately does not attempt to scrape 500k records — it proves the pipeline is real end-to-end at a reduced, honest volume, and argues the scaling path below rather than brute-forcing it.

1. Scaling to 500k+ records

This trial ran **single-process, single-machine**, against 5 source categories, producing on the order of hundreds to low thousands of records. Scaling is a **horizontal-worker problem, not a code problem**, because every scraper module (`src/scrapers/*.py`) is already an independent async task with no shared mutable state beyond the entity-resolution log and the output writer — both of which already have an obvious distributed form:

- **Shard by source space.** Each scraper's natural pagination axis becomes a work-partition key: different arXiv categories (`cs.AI`, `cs.LG`, `cs.CL`, `cs.CV`, ...) or start offset ranges per worker; different YC batch ranges or Algolia facet slices per worker; different job-board identities per worker; different news-feed identities per worker.
- **Run M worker processes** (Celery/RQ, or even a plain multiprocessing.Pool / container-per-shard for a trial-adjacent version of "production"), each assigned one shard, all writing through the same dedup layer (§3) so no record is double-counted across workers.
- **Throughput scales linearly with worker count until the source's own rate limit becomes the bottleneck** — at that point adding workers stops helping, since the ceiling is the API's, not ours. Concretely: **GitHub** (tightest real constraint hit) — the `search` endpoint used for paper→repo matching caps at ~30 req/min *per token*, far tighter than the core API's 5,000/hr; the fix is provisioning N tokens and round-robinning workers, giving $N \times 30$ req/min on search linearly. **arXiv** self-polices to ~1 req/3s with no hard cap; scaling means more categories in flight concurrently, not faster requests against one. **Algolia** (YC) has no published limit; this run's self-imposed ~2 req/sec parallelizes cleanly per facet-shard.
- **News/Jobs are the easiest to scale:** adding more RSS feeds or more job board integrations is purely additive — each new source is one more independent, fixed-cadence worker, with no cross-source coordination needed beyond the shared dedup store.

2. 413 / 429 handling — actual tested behavior

`src/llm/orchestrator.py` implements the 3-tier fallback chain (Gemini → Groq → DeepSeek) with two distinct, deliberately different responses to the two failure modes, both proven live via `python -m src.llm.orchestrator`'s offline simulation (see that module's `__main__` block — it monkeypatches each provider call and asserts the exact fallback order):

- **429 (rate limited):** retried on the **same provider**, exponential backoff + jitter via `tenacity.wait_random_exponential(multiplier=1, max=16)`, capped at **4 attempts** (`stop_after_attempt(4)`) — spreading retries across roughly a 1–16 second window before giving up on that provider and falling through to the next one in the chain.
 - **413 (payload too large): no same-provider retry** — falls through to the next provider immediately, since retrying an oversized payload against the same provider can't succeed. The chunker (`src/llm/chunker.py`) is applied *before* the first attempt, at a configurable `max_tokens` budget (default 3000), using a paragraph-density scorer rather than naive head-truncation: it keeps lead paragraphs, paragraphs with high numeric density (stats/dates), and paragraphs with apparent named entities, while dropping boilerplate (`length < 20` chars, e.g. "Share"/"Subscribe") and heavily penalizing very long blocks (`length > 2000` chars, likely legal/cookie text). Tested live in this run: a synthetic 4,462-token article truncated to a 60-token budget correctly retained the lead sentence and the highest-density statistic paragraph while dropping filler and boilerplate.
 - **GitHub search-endpoint throttling** (a real 429-adjacent constraint discovered during this run, distinct from the LLM orchestrator): initial live testing surfaced that GitHub's `/search/repositories` endpoint has its own ~30 req/min ceiling separate from the core API's 5,000/hr, which the pre-built module hadn't accounted for. Fixed by adding an explicit **2.2s delay between paper-enrichment iterations** (`src/scrapers/arxiv_papers.py`), sized to stay under that endpoint-specific limit — at 1,000 papers this adds ~69 minutes to a full run (observed directly), the binding bottleneck on scaling papers further within a single process — see §1's token round-robin fix.
-

3. Dedup across distributed runs

For a single-process trial run, in-memory dict-keying by natural id (Algolia `objectID`, arXiv `paper_url`, article URL, job listing URL) is sufficient and is what this run uses. At distributed scale, the same principle generalizes to a **shared, cross-worker dedup store**:

- Compute a **content hash** per record — SHA-256 of the canonical `source.url`, or of `source.url + title` where a source's URLs aren't fully stable (some job boards reuse slugs) — before emitting.
 - Check that hash against a **shared set** (Redis `SADD/SISMEMBER` for low-latency membership checks at scale, or a `processed_urls` table with a unique constraint if a relational store is already in play) *before* a worker commits a record.
 - This is what prevents the same article/paper/listing being processed twice when two workers' shards happen to overlap (e.g. a news feed that briefly appears in two category shards, or a job re-posted across boards with the same canonical URL).
 - Entity-level dedup (the *company*, not the *record*) is handled separately and is already implemented and tested in this trial: `EntityCanonicalizer` (`src/resolution/canonicalizer.py`) exact-matches against a seed list first, then fuzzy-matches via `rapidfuzz` (threshold 82.0, tuned against the real "OpenAI" vs "OpenAI" typo case scoring 83.3), logging every decision's method and confidence to the Entity Mapping Log output tab so a human can audit fuzzy merges after the fact.
-

4. Storage justification

This trial: flat JSONL + CSV per entity type (`src/output/writer.py`), chosen deliberately over SQLite/Postgres because the trial's volume (hundreds to low thousands of records, one run, no concurrent writers) doesn't need a database's concurrency guarantees, and JSONL keeps every record trivially diffable, greppable, and re-import-able

without any schema migration ceremony. Every record is validated against its pydantic schema before being written; anything that fails validation is rejected and logged to a `*_rejected.jsonl` file rather than silently dropped, so nothing disappears unaccounted for.

At 500k+ scale, this stops being sufficient for two independent reasons — concurrent-write safety across distributed workers, and the actual shape of the "Intelligence Graph" relationships the brief's own name implies:

- **Postgres** as the relational core: gives ACID writes for concurrent workers, a real unique-constraint-backed dedup table (§3), and **JSONB** columns for the fields that genuinely vary in shape by entity type (e.g. a startup's tags/industries array, a job's raw source payload) without forcing a rigid wide-table schema for data that's naturally semi-structured.
 - **pgvector** (staying inside Postgres) if semantic search over paper abstracts / article text becomes a requirement — avoids standing up a separate vector store for a feature that's naturally an extension of the relational store already in place.
 - **A dedicated graph DB (Neo4j)** is the stronger argument if the "Intelligence Graph" framing is taken literally: the real value in this dataset is the *relationships* — founders who wrote papers, papers cited by other founders' companies, employees who moved between the startups in the Entity Mapping Log. Queries like "papers by authors who founded YC companies" are multi-hop graph traversals that a relational join chain handles increasingly poorly as hop count grows, while a graph DB makes them a first-class, indexed query pattern. For this trial's flat entity-type outputs (no yet-built cross-entity relationship extraction), Postgres+JSONB is the pragmatic default; Neo4j becomes the right call once relationship extraction between entity types is actually built out.
-

5. Known judgment calls (carried from README.md)

- `PricingModel` has 5 values, not the brief's 4 (UNKNOWN added) — YC's directory never exposes pricing; forcing a guess into one of 4 real values would be exactly the hallucination the brief disqualifies for.
 - Fuzzy match threshold is 82.0, tuned against a real typo case, not a round number — see §3 above. The scorer itself was fixed after scaling to 1,000 startups: `wRatio`'s token-overlap heuristic inflated scores for short names sharing a generic word (46 companies falsely merged into "Mistral AI" alone). Switched to plain `fuzz.ratio` + a 6-char minimum-length guard — fuzzy matches dropped 187→25, bulk-collision pattern gone. Full detail in README.md.
 - Papers with Code was considered as an additional source alongside Arxiv; Arxiv+GitHub was used since it produces equivalent schema output for this pipeline's purposes.
 - The GitHub paper→repo matcher went through three rounds of live-testing fixes, each surfaced by manually sense-checking matches rather than trusting the first "it ran without erroring" result: 1. `sort=stars` search matched the same aggregator/daily-tracker repo to 3 unrelated papers in one run — fixed by rejecting aggregator naming patterns, repos with no primary language, and requiring the title verbatim in the fetched README, not just search relevance. 2. That same `sort=stars` ordering buried a paper's genuine brand-new 0-star repo below older wrong aggregator repos — dropping it for default relevance ordering recovered those matches. 3. Relevance ordering then surfaced spam "profile README" repos (`name==owner`) padded with paper titles, plus one repo whose README opened "AUTO-GENERATED BY PAPERFLOW. DO NOT EDIT MANUALLY." — fixed with an `owner==reponame` rejection and a content-level auto-generation check, deliberately content- not name-based. Final: 321/1,000 papers (32.1%) matched, 679 left `null` — consistent with the 37.7% rate at the earlier 300-paper run. Trades recall for precision deliberately (TRD §2.2: a wrong repo is worse than `null`), and the three rounds are evidence precision was verified, not assumed.
-

6. Anti-bot strategy — not demonstrated, by design

Every source here (YC/Algolia, arXiv, GitHub, 5 RSS feeds, RemoteOK + 4 job-board APIs) is first-party and non-adversarial — anti-bot fights were avoided by source selection, not bypass tooling. TRD §6 separately invites demonstrating one Playwright scrape against a JS-heavy, anti-bot-hardened source to prove that capability exists; **this was not built or exercised here**, stated plainly rather than left for a reviewer to notice. It would be a small addition (one scraper behind the same interchangeable-fetcher pattern `jobs_other.py` already uses), not a redesign, if needed.